fieldborders: agricultural field boundaries from Sentinel-2 time series

Tutorial, version 1.0.0

Ehsan Rahimi and Chuleui Jung

Contents

Get started with fieldborders **2**
- A Sentinel-2 time series . . . 2
- Edge and cropland maps . . . 3
- From maps to fields . . . 3
- How good is it? . . . 5
- Adding a deep model . . . 6
- Where next . . . 7

Train a model on your own reference fields **8**
- Reference raster . . . 8
- Which features show the borders? . . . 8
- Training . . . 10
- Thresholds . . . 10
- Saving the model . . . 11
- A fusion for your region . . . 11

Map a new area from Sentinel-2 **12**
- Download a monthly time series . . . 12
- Look at the images . . . 12
- Fields from the pretrained model . . . 13
- Adding the deep model . . . 15
- Export . . . 16
- Tips . . . 16

Gallery: fields around the world **17**
- What the maps show . . . 18
- Imperfect scenes . . . 18
- Data . . . 19

How accurate is it? **20**
- The object metric . . . 20
- Uncertainty and comparisons . . . 20
- The validation study . . . 21
- The pretrained models . . . 23

Get started with fieldborders

A field border is often hard to see in one satellite image: two fields of the same crop can look alike on any given day. Over a season the border becomes obvious, because neighbouring fields are sown, irrigated, cut and harvested at different times. The **temporal boundary contrast** (TBC) method in `fieldborders` uses this. For pairs of pixels on either side of a known border and pairs inside one field, it measures how differently every band and normalised difference changes between the two pixels over a year of monthly Sentinel-2 images, ranks the features by how well this contrast separates the two kinds of pairs, and learns from the best of them:

- an **edge map**, the probability that a pixel lies on a border between two fields, and
- a **cropland map**, the probability that a pixel belongs to a field at all.

Field objects are then grown from seeds inside fields by a marker-controlled watershed on the edge map. The maps can be fused with the output of a deep segmentation model, and the result is scored with the object metric of the Fields of The World (FTW) benchmark.

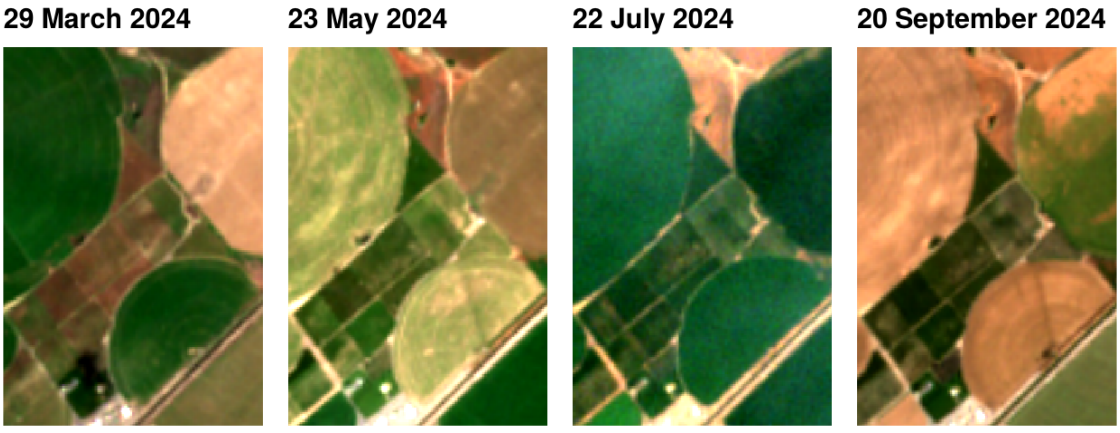
```
library(fieldborders)
library(ggplot2)
library(patchwork)
```

A Sentinel-2 time series

The package ships four small windows (about 1 km across) of monthly Sentinel-2 scenes, ten bands each, on the FTW grid of about 5 m. This one is in the Columbia Basin of Washington, with centre-pivot and rectangular irrigated fields:

```
s <- fb_example_stack("washington")
s
#> <fb_stack> 192 x 192 pixels, 11 dates, 10 bands
#>   dates : 2024-01-14 to 2024-12-09
#>   bands : B02, B03, B04, B05, B06, B07, B08, B8A, B11, B12
#>   scale : 10000
#>   crs   : WGS 84
#>   clear : 100% 100% 100% 100% 100% 100% 100% 100% 100% 100% 100%
```

```
months <- c(3, 5, 7, 9)
wrap_plots(lapply(months, function(k) fb_plot_rgb(s, date = k) + labs(subtitle = NULL)), nrow =
 ↪  1)
```



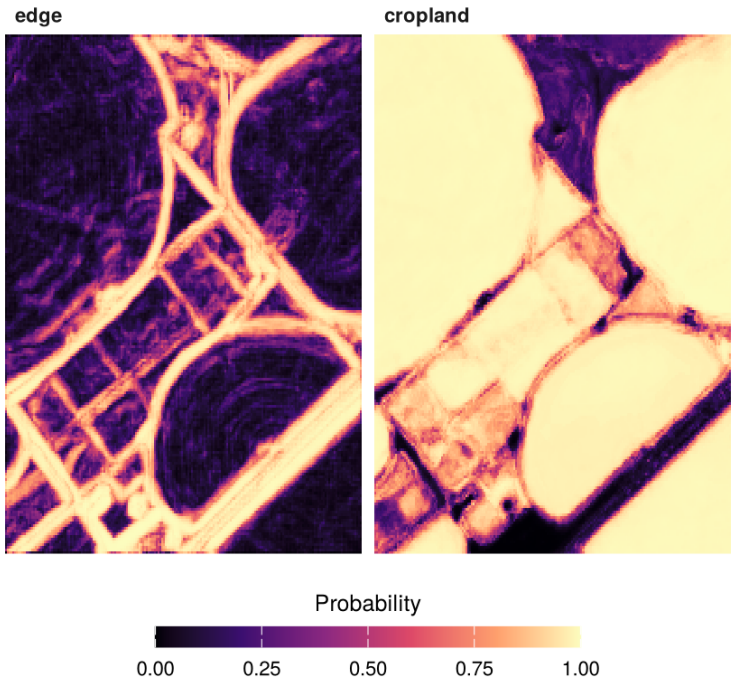
Your own images enter the same way through `fb_read_stack()` (one multi-band file per date) or `fb_stack()` (a `SpatRaster`), and `fb_s2_monthly()` downloads a monthly series for any area (see `vignette("new-area")`).

Edge and cropland maps

`fb_model_pretrained()` returns models fitted on reference fields in California and Washington. `fb_tbc_maps()` applies a model to a stack:

```
m <- fb_model_pretrained("general")
m
#> <fb_model> temporal boundary contrast
#>   bands            : B02, B03, B04, B05, B06, B07, B08, B8A, B11, B12
#>   edge features    : ND(B03,B08) | ND(B02,B07) | B04 | ND(B04,B11) | ND(B03,B11) |
 ↪  ND(B08,B11) | B08 | ND(B02,B05)
#>   cropland learner : xgboost
#>   training         : 75 tiles, 467,670 pairs (167,670 across), 1,470,140 pixels
#>   in-sample AUC    : edge 0.902, cropland 0.968
#>   thresholds       : t_edge 0.60, t_field 0.30, min_size 400 pixels
maps <- fb_tbc_maps(s, m)
```

```
fb_plot_maps(maps)
```



From maps to fields

`fb_delineate()` turns the maps into field objects, with the thresholds that were tuned for the model on separate reference tiles:

```
m$thresholds
#> $t_edge
#> [1] 0.6
#>
#> $t_field
#> [1] 0.3
#>
#> $min_size
#> [1] 400
```

```
#>
#> $min_area
#> [1] 10940
fields <- fb_delineate(maps, thresholds = m$thresholds)
```

```
fb_plot_fields(fields, s, date = 7)
```



`fb_polygons()` converts the objects to an `sf` data frame with the area, perimeter, compactness and mean map values of each field, ready to be written to any vector format:

```
p <- fb_polygons(fields, maps)
head(p)
#> Simple feature collection with 6 features and 6 fields
#> Geometry type: POLYGON
#> Dimension:     XY
#> Bounding box:  xmin: -119.1713 ymin: 46.53552 xmax: -119.161 ymax: 46.54525
#> Geodetic CRS:  WGS 84
#>   field_id   area_ha perimeter_m compactness mean_edge mean_cropland
#> 1        1 23.211101   2571.4177   0.4411237 0.2024311     0.9759057
#> 2        4  1.933387    873.2404   0.3186112 0.5731383     0.8261993
#> 3        8  7.236378   1645.0143   0.3360406 0.3947174     0.9262362
#> 4       12 12.161893   1993.7818   0.3844641 0.2700907     0.9477701
#> 5       13  1.585351    863.4161   0.2672360 0.4966539     0.9203941
#> 6       14  3.386969   1124.9640   0.3363132 0.4967495     0.8474157
#>                         geometry
#> 1 POLYGON ((-119.1713 46.5452...
```

```
#> 2 POLYGON ((-119.1702 46.5384...
#> 3 POLYGON ((-119.167 46.54076...
#> 4 POLYGON ((-119.1636 46.5395...
#> 5 POLYGON ((-119.1666 46.5430...
#> 6 POLYGON ((-119.1651 46.5417...
f <- tempfile(fileext = ".gpkg")
sf::st_write(p, f, quiet = TRUE)
```

`fb_fields()` runs the same steps in one call.

How good is it?

The California example comes with reference fields from the California Department of Water Resources. `fb_reference()` rasterises them to the image grid (field interior, boundary, background), and `fb_evaluate()` matches delineated and reference fields one to one at an intersection over union above 0.5, as in the FTW benchmark:

```
s_ca <- fb_example_stack("california")
f_ca <- fb_example_files("california")
ref_poly <- sf::st_read(f_ca$fields, quiet = TRUE)
ref <- fb_reference(ref_poly, s_ca)
maps_ca <- fb_tbc_maps(s_ca, m)
fields_ca <- fb_delineate(maps_ca, thresholds = m$thresholds)
ev <- fb_evaluate(ref, fields_ca)
fb_accuracy(ev)[, c("reference_fields", "predicted_fields", "obj_precision", "obj_recall",
 ↪  "obj_f1")]
#>   reference_fields predicted_fields obj_precision obj_recall    obj_f1
#> 1               11               13     0.7692308  0.9090909 0.8333333
```

```
fb_plot_fields(fields_ca, s_ca, date = 7, reference = ref_poly)
```



The dashed white lines are the reference fields. A window this small holds only a few fields, so these numbers say little about accuracy in general; `vignette("validation")` summarises the validation on several thousand reference fields.

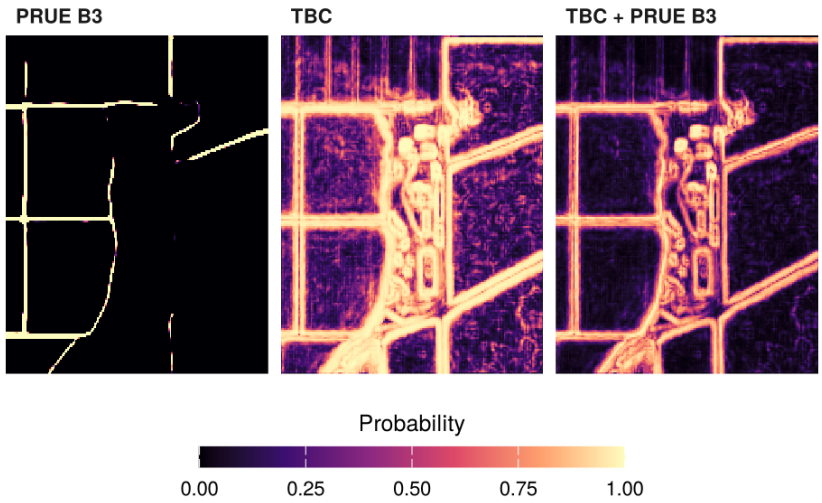
Adding a deep model

Deep segmentation models such as the FTW PRUE models predict, for every pixel, the probability of background, field interior and field boundary from two images of the season. `fb_prue_predict()` runs PRUE in R (it needs the `torch` package); the examples include its output for each window:

```
dl <- fb_read_probabilities(f_ca$prue)
fu <- fb_model_pretrained("general", "fusion")
fused <- fb_fuse(maps_ca, dl, fu)
```

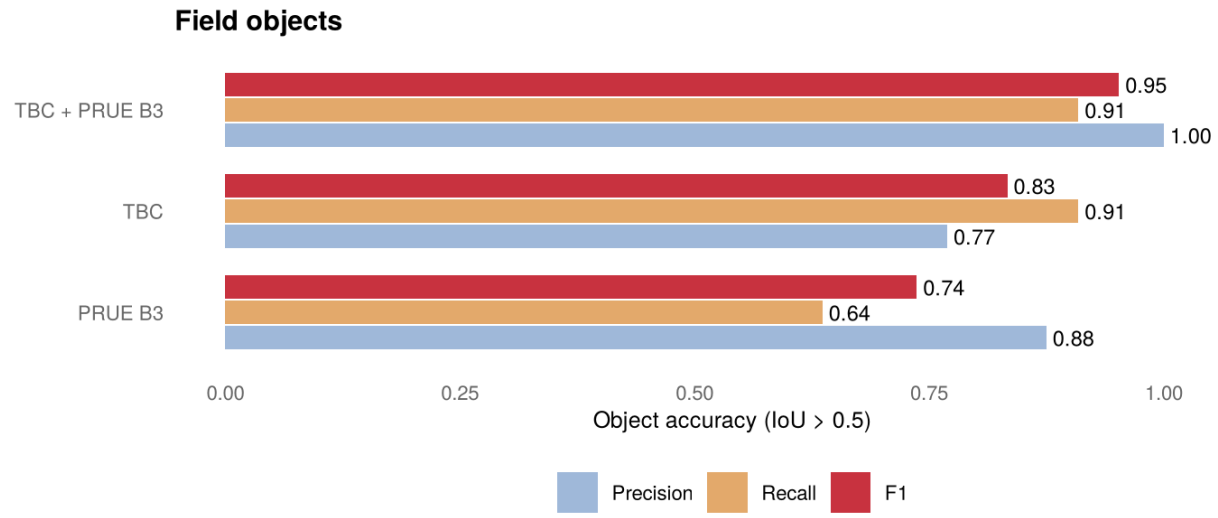
The fusion combines the deep-model and TBC probabilities with two logistic regressions fitted on reference pixels. The three edge maps side by side:

```
edges <- c(dl[["boundary"]], maps_ca[["edge"]], fused[["edge"]])
names(edges) <- c("PRUE B3", "TBC", "TBC + PRUE B3")
fb_plot_maps(edges)
```



and the accuracy of the objects obtained from each, all with the same object procedure (the deep model alone with the thresholds of the fusion):

```
counts <- rbind(
  cbind(method = "PRUE B3", fb_evaluate(ref, fb_delineate(dl, thresholds = fu$thresholds))),
  cbind(method = "TBC", fb_evaluate(ref, fields_ca)),
  cbind(method = "TBC + PRUE B3", fb_evaluate(ref, fb_delineate(fused, thresholds =
    ↪  fu$thresholds))))
fb_plot_accuracy(fb_accuracy(counts, by = "method"))
```



Where next

- `vignette("training")`: train a model on your own reference fields and choose the features and thresholds.
- `vignette("new-area")`: download Sentinel-2 for any area and map its fields.
- `vignette("gallery")`: the pretrained model in six farming regions around the world.
- `vignette("validation")`: the accuracy measures and the results of the validation.

Train a model on your own reference fields

The pretrained models were fitted on irrigated and dryland agriculture in California and Washington. Where reference fields exist for your region, train a model on them, alone or together with other regions, and compare it with the pretrained models on separate tiles: in the validation, a model fitted on California and Washington together was at least as accurate on Washington test tiles as one fitted on Washington alone. This article trains a model on the California example. The window is small and the same fields are used to train and to evaluate, so the code shows the steps, not the accuracy to expect; with real data, fit, tune and test on different tiles, as in the validation of the package (`vignette("validation")`).

```
library(fieldborders)
library(ggplot2)
s <- fb_example_stack("california")
fields_poly <- sf::st_read(fb_example_files("california")$fields, quiet = TRUE)
```

Reference raster

`fb_reference()` burns the reference polygons into the image grid: field interiors are class 1, polygon outlines class 2 and everything else background (class 0):

```
ref <- fb_reference(fields_poly, s)
terra::freq(ref$class)
#>   layer value count
#> 1     1     0  7691
#> 2     1     1 27370
#> 3     1     2  1803
```

When only some fields of an area are mapped (one crop, say), pixels outside the polygons are not known to be background. Use `presence_only = TRUE`: they become class 3, which training ignores and which does not count against a delineation as false positives.

Which features show the borders?

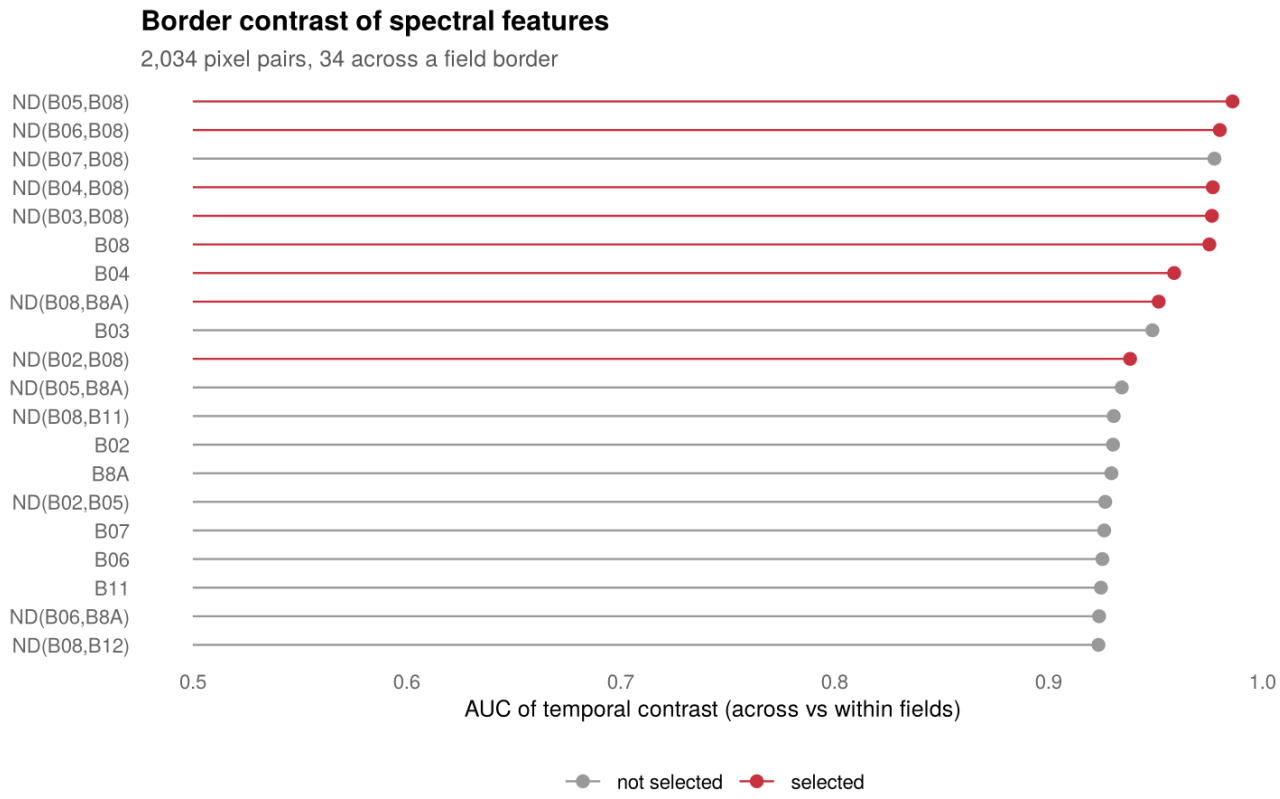
Every candidate feature, by default the ten bands and their 45 normalised differences, is compared between the two ends of pixel pairs that straddle a border between two fields (*across* pairs) and of pairs inside one field (*within* pairs). For each pair the contrast `log(|f(p) - f(q)| + 1e-4)` is computed on every date that is clear at both ends, and its maximum and mean over the season are kept. `fb_rank_contrast()` ranks the features by the area under the ROC curve with which the seasonal maximum separates across from within pairs:

```
rk <- fb_rank_contrast(s, ref, n_pairs = 2000)
rk
#> <fb_contrast> 2034 pairs ( 34 across ), 55 features, 1 strata
#>       feature type   auc
#>  ND(B05,B08)   nd 0.986
#>  ND(B06,B08)   nd 0.980
#>  ND(B07,B08)   nd 0.978
#>  ND(B04,B08)   nd 0.977
#>  ND(B03,B08)   nd 0.976
#>          B08 band 0.975
#>          B04 band 0.959
#>  ND(B08,B8A)   nd 0.951
#>          B03 band 0.949
#>  ND(B02,B08)   nd 0.938
```

`fb_select_features()` walks down the ranking and skips features that are strongly correlated (Spearman's rho above 0.9) with one already chosen:

```
sel <- fb_select_features(rk, k = 8)
sel
#> [1] "ND(B05,B08)" "ND(B06,B08)" "ND(B04,B08)" "ND(B03,B08)" "B08"
#> [6] "B04"         "ND(B08,B8A)" "ND(B02,B08)"
```

```
fb_plot_ranking(rk, n = 20, selected = sel)
```



Here normalised differences between the red-edge and near-infrared bands lead: they follow canopy development, which differs between neighbouring fields through the season.

Published spectral indices

With the `svid` package installed, the candidates can include the published indices of its catalogue, rewritten in Sentinel-2 band names, and expressions generated by an `svid` grammar:

```
fs <- fb_feature_set(fb_example_bands(), set = "svid")
table(fs$type)
#>
#> band   nd svid
#>   10   45  193
rk_svid <- fb_rank_contrast(s, ref, features = fs, n_pairs = 2000)
head(rk_svid$ranking[, c("feature", "type", "auc")], 10)
#>        feature type       auc
#> 1  ND(B05,B08)   nd 0.9859853
#> 2   svid:NDREI svid 0.9859853
#> 3    svid:GARI svid 0.9839853
#> 4  svid:GOSAVI svid 0.9801324
#> 5  ND(B06,B08)   nd 0.9800588
#> 6  ND(B07,B08)   nd 0.9775441
```

```
#> 7  ND(B04,B08)   nd 0.9767647
#> 8    svid:IPVI svid 0.9767353
#> 9    svid:NDVI svid 0.9767353
#> 10     svid:PI svid 0.9767353
```

Training

`fb_train()` fits the edge model, a logistic regression on the seasonal maximum and mean contrasts of the selected features, and the cropland model, gradient-boosted trees (or a random forest with `learner = "ranger"`, or a logistic regression with `learner = "glm"`) on per-pixel temporal percentiles of the bands and three indices. Passing the ranking avoids computing the pair contrasts twice:

```
m <- fb_train(s, ref, contrast = rk, k = 8, n_pixels = 5000, seed = 1)
m
#> <fb_model> temporal boundary contrast
#>   bands            : B02, B03, B04, B05, B06, B07, B08, B8A, B11, B12
#>   edge features    : ND(B05,B08) | ND(B06,B08) | ND(B04,B08) | ND(B03,B08) | B08 | B04 |
 ↪  ND(B08,B8A) | ND(B02,B08)
#>   cropland learner : xgboost
#>   training         : 1 tiles, 2,034 pairs (34 across), 10,000 pixels
#>   in-sample AUC    : edge 0.999, cropland 1.000
#>   thresholds       : t_edge 0.50, t_field 0.50, min_size 50 pixels
```

Several tiles are passed as lists. For many large tiles, the list elements can be functions that load a tile when it is needed, so that only one tile is in memory at a time; `strata` averages the feature ranking over regions or field types:

```
# one folder of monthly images (file names with dates) and one GeoPackage of fields per tile
tiles <- c("tile_01", "tile_02", "tile_03")
images <- function(t) list.files(file.path("s2", t), pattern = "\\.tif$", full.names = TRUE)
load_stack <- function(t) function() fb_read_stack(images(t), bands = fb_example_bands())
load_ref <- function(t) function() {
  fb_reference(sf::st_read(file.path("labels", paste0(t, ".gpkg")), quiet = TRUE),
   ↪  terra::rast(images(t)[1]))
}
m <- fb_train(lapply(tiles, load_stack), lapply(tiles, load_ref), strata = c("irrigated",
 ↪  "irrigated", "dryland"))
```

Thresholds

Three thresholds turn the maps into fields: seeds are pixels with cropland probability above `t_field` and edge probability below `t_edge`, and objects smaller than `min_size` pixels are dropped. `fb_tune()` evaluates every combination on tiles with reference fields and returns them sorted by object F1:

```
maps <- fb_tbc_maps(s, m)
tn <- fb_tune(maps, ref, t_edge = c(0.2, 0.3, 0.4, 0.5, 0.6), t_field = c(0.5, 0.7, 0.9),
              min_size = c(0, 50, 100, 200))
head(tn)
#>   t_edge t_field min_size tp fp fn precision    recall       f1
#> 1    0.2     0.5       50 10  0  1         1 0.9090909 0.952381
#> 2    0.2     0.5      100 10  0  1         1 0.9090909 0.952381
#> 3    0.2     0.5      200 10  0  1         1 0.9090909 0.952381
#> 4    0.3     0.5       50 10  0  1         1 0.9090909 0.952381
#> 5    0.3     0.5      100 10  0  1         1 0.9090909 0.952381
#> 6    0.3     0.5      200 10  0  1         1 0.9090909 0.952381
m$thresholds <- as.list(tn[1, c("t_edge", "t_field", "min_size")])
```

Choose the thresholds on tiles that are not used to fit the model, and report accuracy on a third set of tiles.

```
fb_plot_fields(fb_delineate(maps, thresholds = m$thresholds), s, date = 7, reference =
 ↪  fields_poly)
```



Saving the model

```
f <- tempfile(fileext = ".rds")
fb_save_model(m, f)
m2 <- fb_load_model(f)
identical(m2$edge, m$edge)
#> [1] TRUE
```

A fusion for your region

With deep-model probabilities on the training tiles, `fb_fit_fusion()` fits the combination of the two models on the same reference rasters:

```
dl <- fb_read_probabilities(fb_example_files("california")$prue)
fu <- fb_fit_fusion(maps, dl, ref, seed = 1)
fu
#> <fb_fusion> logistic fusion of deep-model and TBC maps
#>   boundary: 0.020 + 0.114 logit P(boundary) + 0.207 logit(edge)
#>   field   : 2.606 + 0.090 logit P(field + boundary) + 0.782 logit(cropland)
#>   thresholds: t_edge 0.40, t_field 0.50, min_size 50 pixels
```

Map a new area from Sentinel-2

This article maps the fields of an area for which the package has no data: about 1.6 by 1.6 km near Jovein, Khorasan Razavi, Iran, with a centre pivot and many small rectangular plots, in 2024. Downloading needs the `rstac` package and an internet connection, and the deep model needs `torch`, so the code is not run when the article is built. The output and the figures below were produced with this code; the script that made them, `data-raw/vignette_new_area.R`, is in the source repository.

Download a monthly time series

`fb_s2_monthly()` searches the Microsoft Planetary Computer catalogue for Sentinel-2 Level-2A scenes of the area. Among the four scenes of each month with the least cloud cover, it keeps the one with the fewest cloudy, shadowed or missing pixels in the area, and resamples its ten bands to one grid in the UTM zone of the area, at 5 m by default, the pixel size of the pretrained models:

```
library(fieldborders)
aoi <- c(57.436, 36.606, 57.454, 36.620)   # longitude and latitude: west, south, east, north
s <- fb_s2_monthly(aoi, "2024-01-01", "2024-12-31", dir = "jovein_s2")
#> 48 scenes found; reading scene classifications
#> 12 months kept; reading 10 bands
s
#> <fb_stack> 314 x 324 pixels, 12 dates, 10 bands
#>   dates : 2024-01-31 to 2024-12-26
#>   bands : B02, B03, B04, B05, B06, B07, B08, B8A, B11, B12
#>   scale : 10000
#>   crs   : WGS 84 / UTM zone 40N
#>   clear : 100% 100% 100% 100% 100% 100% 100% 100% 100% 100% 100% 100%
```

The download took 2.4 minutes. With `dir`, the scenes are written as GeoTIFFs with an index table, so the series can be read again later without downloading:

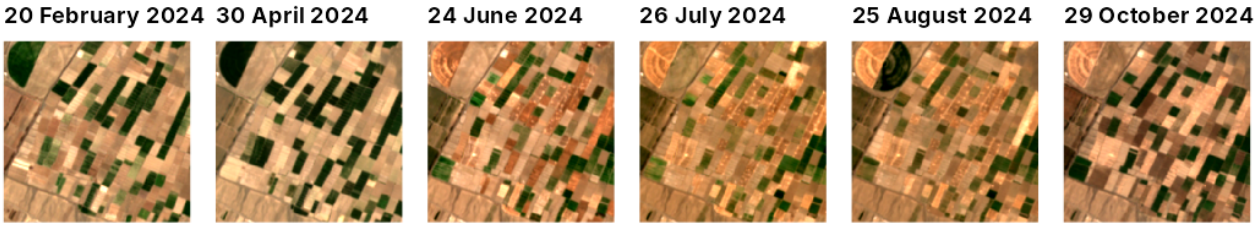
```
idx <- read.csv("jovein_s2/s2_index.csv")
s <- fb_read_stack(idx$image, bands = fb_example_bands(), dates = idx$date, clear = idx$clear)
```

The scene identifiers are kept with the stack, for the record of a study:

```
attr(s, "items")[1:3]
#> [1] "S2A_MSIL2A_20240131T070121_R063_T40SEF_20240131T113906"
#> [2] "S2A_MSIL2A_20240220T065921_R063_T40SEF_20240220T113933"
#> [3] "S2B_MSIL2A_20240323T064629_R020_T40SEF_20240323T103903"
```

Look at the images

```
library(patchwork)
wrap_plots(lapply(c(2, 4, 6, 7, 8, 10), function(k) fb_plot_rgb(s, date = k)), nrow = 1)
```



The scene classification that marks clear pixels misses some clouds, haze and shadows, and it treats snow as clear (add class 11 to `invalid_classes` to change this). If a date is visibly affected, drop it before mapping, for example `s <- fb_subset_dates(s, fb_dates(s) != as.Date("2024-02-20"))`. TBC uses the seasonal maximum and mean of its contrasts, so it needs no fixed number of dates, but more dates help: in the California validation, object F1 was 0.53 with two dates, 0.56 with four and 0.57 with twelve.

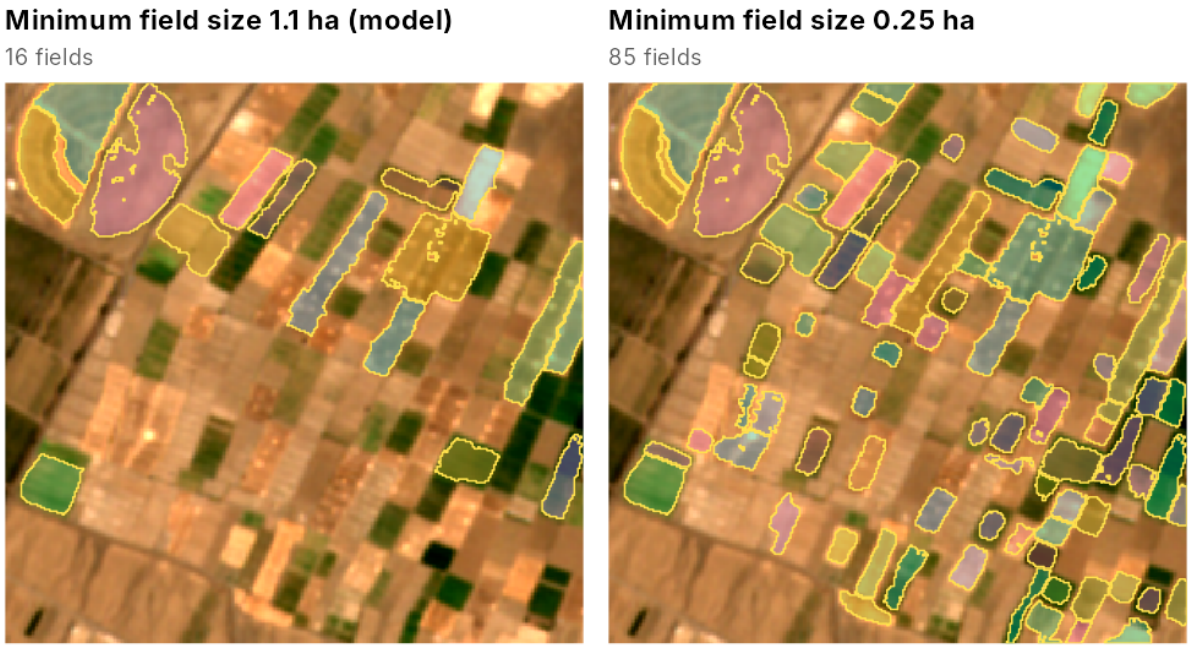
Fields from the pretrained model

```
res <- fb_fields(s)          # pretrained "general" model and its thresholds
res$polygons
#> Simple feature collection with 16 features and 6 fields
#> Geometry type: POLYGON
#> Dimension:     XY
#> Bounding box:  xmin: 539020 ymin: 4051600 xmax: 540605 ymax: 4052820
#> Projected CRS: WGS 84 / UTM zone 40N
#> First 5 features:
#>   field_id area_ha perimeter_m compactness mean_edge mean_cropland
#> 1        1  3.0325        1100   0.3149382 0.4988499     0.9299231
#> 2        2  1.9325         690   0.5100716 0.4472115     0.8929288
#> 3        3  3.9800        1220   0.3360263 0.4634693     0.9150371
#> 4        7  6.5050        2330   0.1505724 0.2882269     0.7485888
#> 5       15  2.2350         830   0.4076911 0.5003931     0.9149175
#>                         geometry
#> 1 POLYGON ((539055 4052750, 5...
#> 2 POLYGON ((539085 4051780, 5...
#> 3 POLYGON ((539110 4052820, 5...
#> 4 POLYGON ((539385 4052820, 5...
#> 5 POLYGON ((539470 4052480, 5...
fb_plot_fields(res$fields, s, date = 7, fill_alpha = 0.4)
```



Many plots here are smaller than the minimum field size of the pretrained models, about 1.1 ha (`m$thresholds$min_area`, in square metres). A smaller minimum keeps them, but it also keeps more fragments of fields:

```
m <- fb_model_pretrained("general")
small <- fb_delineate(res$maps, thresholds = m$thresholds, min_area = 2500)
fb_plot_fields(res$fields, s, date = 7, fill_alpha = 0.4, title = "Minimum field size 1.1 ha
↪  (model)") +
  fb_plot_fields(small, s, date = 7, fill_alpha = 0.4, title = "Minimum field size 0.25 ha")
```

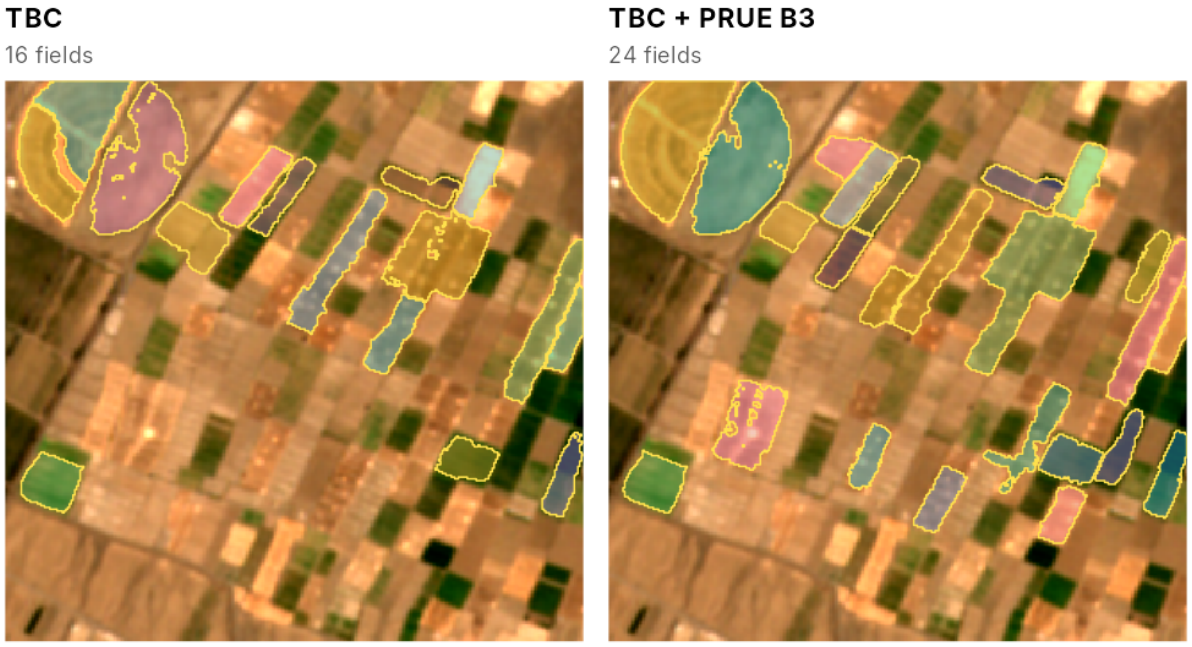


Without reference fields, choose such settings by looking at the result, or better, map a few fields by hand and tune the thresholds on them with `fb_tune()`.

Adding the deep model

`fb_prue_predict()` downloads the PRUE B3 model once (about 54 MB) and runs it on two dates of the stack. The pretrained fusion expects the months used to fit it, March and July; in the southern hemisphere use the same stage of the season, for example September and January.

```
dl <- fb_prue_predict(s, months = c(3, 7))
res2 <- fb_fields(s, dl = dl)
fb_plot_fields(res$fields, s, date = 7, fill_alpha = 0.4, title = "TBC") +
  fb_plot_fields(res2$fields, s, date = 7, fill_alpha = 0.4, title = "TBC + PRUE B3")
```



With the deep model, 24 fields were delineated instead of 16. It added several of the narrow plots, but it also joined plots into one object in the centre, and many small plots are missing from both maps.

Export

```
sf::st_write(res2$polygons, "jovein_fields.gpkg")
terra::writeRaster(res2$maps, "jovein_maps.tif")
```

The polygons carry their area in hectares, perimeter, compactness (4 pi area divided by the squared perimeter) and the mean edge and cropland probabilities inside each field.

Tips

- The deep model works on windows of 256 by 256 pixels (about 1.3 km at 5 m) and pads smaller areas by reflection, so map areas at least that large when using it; TBC works on any size.
- The pretrained models were fitted in irrigated and dryland agriculture of California and Washington. Where reference fields exist, train a local model (`vignette("training")`) and check it on separate tiles (`vignette("validation")`).
- `fb_s2_monthly()` accepts an `sf` polygon, a bounding box or a `SpatRaster` whose grid is then used as is, for example to match an existing map.

Gallery: fields around the world

The pretrained `"general"` model was fitted in California and Washington. These maps show what it produces, without any local training, in six other farming regions: one season of monthly Sentinel-2 scenes downloaded with `fb_s2_monthly()` for a box of about 1.6 by 1.6 km in each region, and the fields delineated by `fb_fields()` with the model's thresholds. There are no reference fields for these boxes, so the maps show how the method behaves, not how accurate it is; for accuracy, see `vignette("validation")`. Downloading needs the `rstac` package and an internet connection, so the code is not run when the article is built. The figure was produced with this code; the script that made it, `data-raw/vignette_gallery.R`, is in the source repository.




```
library(fieldborders)
library(ggplot2)
library(patchwork)

sites <- data.frame(
  label = c("Poltava Oblast, Ukraine", "Pergamino, Argentina", "East Oweinat, Egypt", "Beauce,
   ↪  France",
            "Haft Tappeh, Khuzestan, Iran", "Ludhiana, Punjab, India"),
  lon = c(33.30, -60.45, 28.6584, 1.65, 48.3632, 75.65),
  lat = c(50.30, -33.85, 22.5527, 48.32, 32.0069, 30.85),
  start = c("2024-01-01", "2023-07-01", "2024-01-01", "2024-01-01", "2024-01-01", "2024-01-01"),
  end = c("2024-12-31", "2024-06-30", "2024-12-31", "2024-12-31", "2024-12-31", "2024-12-31"))

# a box of about 1.6 km around a point, in longitude and latitude
box <- function(lon, lat, half_km = 0.8) {
  dlat <- half_km / 111.32
```

```
  dlon <- half_km / (111.32 * cos(lat * pi / 180))
  c(lon - dlon, lat - dlat, lon + dlon, lat + dlat)
}
# the date with the highest median NDVI, shown as background
greenest <- function(s) {
  nb <- length(s$bands)
  v <- vapply(seq_along(s$dates), function(k) {
    x <- terra::values(s$data[[(k - 1) * nb + match(c("B08", "B04"), s$bands)]], mat = TRUE)
    median((x[, 1] - x[, 2]) / (x[, 1] + x[, 2] + 1e-4), na.rm = TRUE)
  }, 0)
  which.max(v)
}

m <- fb_model_pretrained("general")
panels <- lapply(seq_len(nrow(sites)), function(i) {
  s <- fb_s2_monthly(box(sites$lon[i], sites$lat[i]), sites$start[i], sites$end[i])
  res <- fb_fields(s, m)
  k <- greenest(s)
  fb_plot_fields(res$fields, s, date = k, fill_alpha = 0.35, title = sites$label[i]) +
    labs(subtitle = sprintf("%s, %d fields", format(s$dates[k], "%B %Y"), nrow(res$polygons)))
})
wrap_plots(panels, ncol = 3)
```

In the southern hemisphere (Pergamino) the season runs from July to June. Each download took 1.4 to 3.0 minutes.

Site	Dates	Date shown	Fields	Median field area (ha)
Poltava Oblast, Ukraine	9	17 July 2024	17	3.1
Pergamino, Argentina	12	26 January 2024	13	19.3
East Oweinat, Egypt	12	31 January 2024	11	4.4
Beauce, France	6	5 March 2024	32	4.3
Haft Tappeh, Khuzestan, Iran	12	25 January 2024	33	3.9
Ludhiana, Punjab, India	12	12 March 2024	16	2.5

Fields cut by the edge of a box are counted and measured only within it.

What the maps show

- **Large fields with different crops** (Pergamino, Beauce, Haft Tappeh) come out as separate objects that follow the field borders. Some fields are cut into pieces or have ragged outlines where the cover within them varies, as in Beauce and in the middle of the Haft Tappeh box.
- **Centre pivots** in East Oweinat are delineated as circles. The rectangular plots with sparse vegetation in the middle of that box are broken into irregular fragments.
- **Neighbouring fields with little contrast between them** are merged: several of the large fields of Poltava form one object.
- **Small plots** in Ludhiana are merged into a few large objects. The borders between them are mostly bunds much narrower than a Sentinel-2 pixel (10 m), which show only where the neighbouring plots differ in cover.

Imperfect scenes

`fb_s2_monthly()` keeps, for each month, the scene with the fewest pixels that the Sentinel-2 scene classification marks as missing, defective, cloud, cloud shadow or cirrus. That classification marked every scene of these six series as clear, but in true colour the January scene of Poltava is covered by snow, the April scene of Poltava and the July, August and October scenes of Ludhiana are partly cloudy or hazy, and the March scene of Beauce holds a small cloud and its shadow. Dropping these dates with `fb_subset_dates()` changed the delineated fields as follows:

Site	Dates dropped	Fields with all dates	Fields without them
Poltava Oblast, Ukraine	12 January, 16 April	17 (of 9 dates)	18 (of 7 dates)
Beauce, France	5 March	32 (of 6 dates)	27 (of 5 dates)
Ludhiana, Punjab, India	30 July, 24 August, 28 October	16 (of 12 dates)	18 (of 9 dates)

The number of fields changed by one to five. In Beauce, which had only six usable months, the fields became more fragmented without the March scene. As a rule of thumb, drop a date that is largely covered by cloud, haze or snow, and keep one in which only a small part is affected. Snow can also be marked as invalid when downloading, with `invalid_classes = c(0, 1, 3, 8, 9, 10, 11)`, but the scene classification can label bright bare ground as snow.

Data

Contains modified Copernicus Sentinel data 2023 and 2024, accessed through the Microsoft Planetary Computer.

How accurate is it?

```
library(fieldborders)
library(ggplot2)
```

The object metric

Field boundaries are judged by whether each field comes out as one object. `fb_evaluate()` uses the object metric of the Fields of The World (FTW) benchmark:

- reference fields are the 4-connected shapes of the field-interior class of the reference raster, and delineated fields the objects of `fb_delineate()`;
- a delineated and a reference field match when their intersection over union (IoU) exceeds 0.5. Above 0.5 a field can match at most one other, so the matching is one to one without any assignment step;
- object precision is the share of delineated fields that match, object recall the share of reference fields that are matched, and F1 their harmonic mean. Delineated fields that lie mostly (more than half) in unlabelled reference pixels are not counted as false positives, so that partial reference maps can be used.

`fb_evaluate()` returns counts, which add up over tiles; `fb_accuracy()` turns them into rates, including the recall of reference fields by size class:

```
s <- fb_example_stack("california")
f <- fb_example_files("california")
ref <- fb_reference(sf::st_read(f$fields, quiet = TRUE), s)
m <- fb_model_pretrained("general")
fields <- fb_delineate(fb_tbc_maps(s, m), thresholds = m$thresholds)
ev <- fb_evaluate(ref, fields)
ev[, c("tp", "fp", "fn", "obj_tp", "obj_fp", "obj_fn", "n_pred")]
#>      tp   fp  fn obj_tp obj_fp obj_fn n_pred
#> 1 26376 2722 994     10      3      1     13
fb_accuracy(ev)[, c("reference_fields", "predicted_fields", "obj_precision", "obj_recall",
 ↪  "obj_f1")]
#>   reference_fields predicted_fields obj_precision obj_recall    obj_f1
#> 1               11               13     0.7692308  0.9090909 0.8333333
```

Summing counts over tiles before computing the rates, as `fb_accuracy()` does, weights every field equally, whatever the tile it lies in.

Uncertainty and comparisons

Fields in one tile share imagery, crops and management, so tiles, not fields, are the units to resample. `fb_bootstrap()` draws tiles with replacement (within strata, if given) and uses the same draws for the two methods of a comparison. To show the interface, the California example is cut into four quarters, standing in for tiles:

```
quarters <- function(r) {
  e <- as.vector(terra::ext(r)); xm <- mean(e[1:2]); ym <- mean(e[3:4])
  list(terra::ext(e[1], xm, ym, e[4]), terra::ext(xm, e[2], ym, e[4]),
       terra::ext(e[1], xm, e[3], ym), terra::ext(xm, e[2], e[3], ym))
}
dl <- fb_read_probabilities(f$prue)
fu <- fb_model_pretrained("general", "fusion")
methods <- list(TBC = fields,
                `TBC + PRUE B3` = fb_delineate(fb_fuse(fb_tbc_maps(s, m), dl, fu), thresholds =
                 ↪  fu$thresholds))
counts <- do.call(rbind, lapply(seq_along(quarters(ref)), function(k) {
  e <- quarters(ref)[[k]]
```

```
  do.call(rbind, lapply(names(methods), function(nm)
    cbind(tile = k, method = nm, fb_evaluate(terra::crop(ref, e), terra::crop(methods[[nm]],
     ↪  e)))))
}))
fb_accuracy(counts, by = "method")[, c("method", "obj_precision", "obj_recall", "obj_f1")]
#>          method obj_precision obj_recall    obj_f1
#> 1           TBC     0.7142857     0.9375 0.8108108
#> 2 TBC + PRUE B3     1.0000000     0.9375 0.9677419
fb_bootstrap(counts, c("TBC + PRUE B3", "TBC"), B = 1000)
#>            comparison metric difference         lo   hi
#> 1 TBC + PRUE B3 - TBC obj_f1  0.1569311 0.07692308 0.25
```

With four tiles the interval is meaningless; the validation below used 44 to 54 test tiles per region.

The validation study

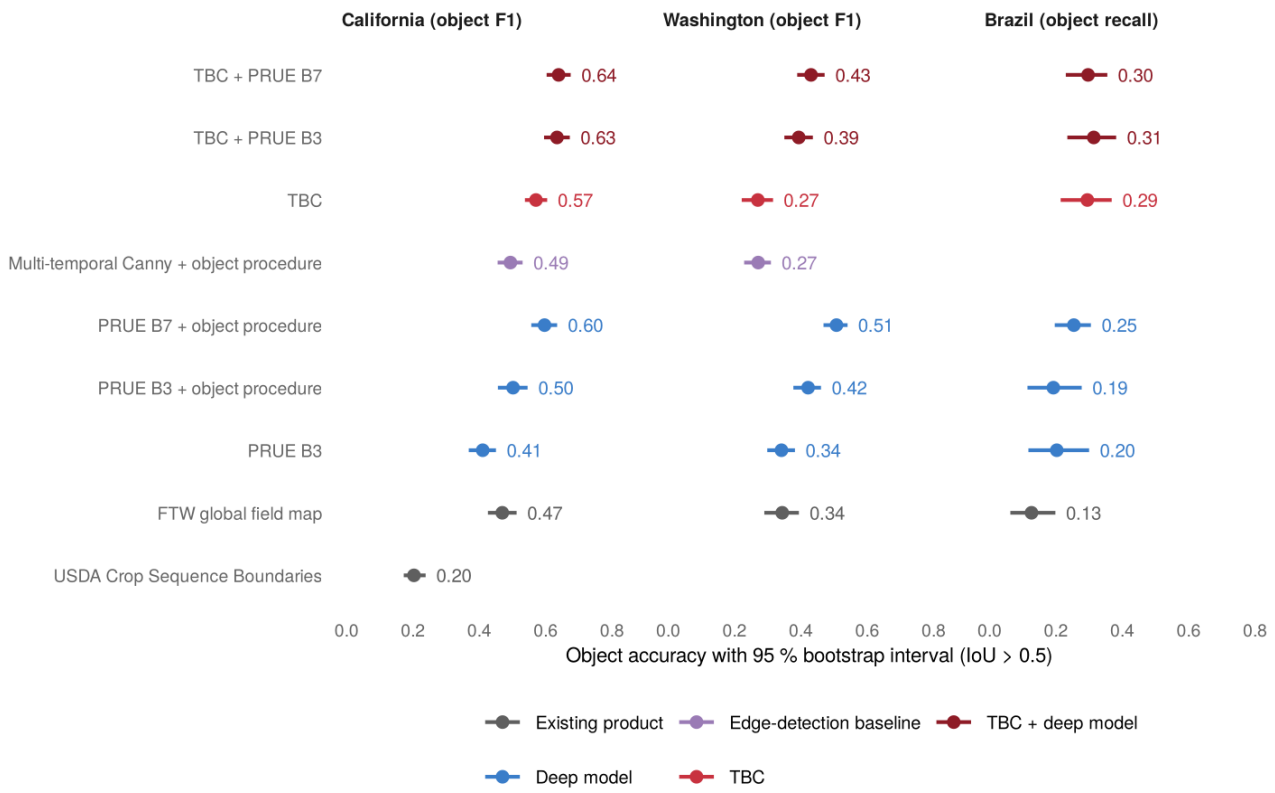
The method was developed in California and validated in three regions on tiles of 512 x 512 pixels (3.1 km north to south and 2.1 to 3.1 km east to west). In each region the tiles were assigned to fitting, tuning and test sets in blocks of 0.2 degrees, so that neighbouring tiles fell into the same set. Models were fitted on the fitting tiles, the input months of the deep model and all thresholds were chosen on the tuning tiles, and the test tiles were evaluated once:

- **California 2024**: reference fields from the California Department of Water Resources Statewide Crop Mapping 2024; 120 tiles in four farming regions (44 fitting, 22 tuning and 54 test tiles, with 3,696 reference fields).
- **Washington**: reference fields from the Washington State Department of Agriculture agricultural land use map, with imagery of 2024 or 2025, the year in which most fields of a tile were surveyed; 90 tiles in the irrigated Columbia Basin, the dryland farming of the Palouse and the Yakima Valley (31 fitting, 15 tuning and 44 test tiles, with 2,155 reference fields). Models were fitted and tuned on Washington tiles.
- **Brazil 2019/20**: reference fields from the crop maps of the Brazilian National Supply Company (CONAB) for summer crops in Tocantins and irrigated rice in Rio Grande do Sul; 60 tiles (13 tuning and 47 test tiles, with 594 reference fields). These maps outline only the mapped crop, so most fields in a tile are unlabelled and object precision says little; object recall is the measure that counts. The models were fitted and tuned on California and Washington tiles, without any Brazilian labels.

The deep models are the FTW PRUE models with EfficientNet-B3 (trained on CC-BY data) and EfficientNet-B7 encoders, applied to two monthly scenes: March and July in California, May and August in Washington (chosen on the tuning tiles) and September and January in Brazil. "Object procedure" means that their probabilities were turned into fields with the same seeds, watershed and size filter as TBC. The FTW global field map and, in California, the USDA Crop Sequence Boundaries were evaluated on the same tiles without any fitting.

```
acc <- read.csv(system.file("extdata", "validation", "object_accuracy.csv", package =
 ↪  "fieldborders"))
va <- acc[acc$analysis == "primary", ]
f1 <- va$primary_metric == "obj_f1"
va$value <- ifelse(f1, va$obj_f1, va$obj_recall)
va$lo <- ifelse(f1, va$f1_lo, va$recall_lo)
va$hi <- ifelse(f1, va$f1_hi, va$recall_hi)
va$panel <- factor(paste(va$region, ifelse(f1, "(object F1)", "(object recall)")),
                   levels = c("California (object F1)", "Washington (object F1)", "Brazil
                    ↪  (object recall)"))
va$method <- factor(va$method, levels = c("USDA Crop Sequence Boundaries", "FTW global field
 ↪  map", "PRUE B3",
                                          "PRUE B3 + object procedure", "PRUE B7 + object procedure",
                                              "Multi-temporal Canny + object procedure", "TBC", "TBC
                                                ↪  + PRUE B3",
                                               "TBC + PRUE B7"))
cols <- c(`Existing product` = "#5F5F5F", `Deep model` = "#3A7DC9", `Edge-detection baseline` =
 ↪  "#9A7BB5",
```

```
          TBC = "#C8323F", `TBC + deep model` = "#8E1B26")
ggplot(va, aes(value, method, colour = family)) +
  geom_errorbar(aes(xmin = lo, xmax = hi), width = 0, linewidth = 0.7, orientation = "y") +
  geom_point(size = 2) +
  geom_text(aes(x = hi, label = sprintf("%.2f", value)), hjust = -0.3, size = 2.6, show.legend =
   ↪  FALSE) +
  facet_wrap(~panel) +
  scale_colour_manual(values = cols, breaks = names(cols), name = NULL) +
  scale_x_continuous(limits = c(0, 0.85), breaks = seq(0, 0.8, 0.2)) +
  labs(x = "Object accuracy with 95 % bootstrap interval (IoU > 0.5)", y = NULL) +
  fb_theme(base_size = 9) +
  theme(legend.position = "bottom", axis.title.x = element_text(size = 8)) +
  guides(colour = guide_legend(nrow = 2))
```



The results differ between the regions:

- In **California**, TBC without any deep model reached an object F1 of 0.57, above the CC-BY PRUE model with or without the object procedure (0.50 and 0.41), the FTW global map (0.47) and the multi-temporal Canny edges given the same cropland map and object procedure (0.49); it did not differ clearly from PRUE B7 with the object procedure (0.60). Fusing TBC with PRUE B3 or B7 gave the highest F1 (0.63 and 0.64).
- In **Washington**, TBC split many fields into fragments (object precision 0.20), most of all in the dryland farming of the Palouse. Its F1 of 0.27 was below that of PRUE B3 and B7 with the object procedure (0.42 and 0.51) and of the FTW global map (0.34), and level with the Canny edges. The F1 of the fusion (0.39 with B3) was higher than that of TBC alone and of the FTW global map, but not higher than that of the deep models with the object procedure.
- In **Brazil**, with models from California and Washington, TBC and its fusion with B3 recovered more reference fields (object recall 0.29 and 0.31) than PRUE B3 with the object procedure (0.19) and the FTW global map (0.13). The gain came from the irrigated rice of Rio Grande do Sul; for the summer crops of Tocantins, recall was similar or lower. All methods recovered fewer than a third of the reference fields.

In Washington, the tuning chose the largest minimum field size of the pre-specified grid (50 pixels, about 0.14 ha) for TBC. In a secondary analysis, added after these results were seen, the thresholds of TBC, PRUE with the object procedure and their fusion were tuned again on the Washington tuning tiles over the wider grid used for the pretrained models (minimum field sizes up to 400 pixels), and the test tiles were evaluated again:

```
ex <- acc[acc$analysis != "primary", ]
ex$`F1 (95 % CI)` <- sprintf("%.2f (%.2f to %.2f)", ex$obj_f1, ex$f1_lo, ex$f1_hi)
ex$Precision <- round(ex$obj_precision, 2)
ex$Recall <- round(ex$obj_recall, 2)
knitr::kable(ex[, c("method", "Precision", "Recall", "F1 (95 % CI)")], row.names = FALSE,
             col.names = c("Method (Washington, wider grid)", "Precision", "Recall", "F1 (95 %
              ↪  CI)"))
```

Method (Washington, wider grid)	Precision	Recall	F1 (95 % CI)
PRUE B3 + object procedure	0.64	0.33	0.44 (0.40 to 0.48)
PRUE B7 + object procedure	0.71	0.41	0.52 (0.48 to 0.55)
TBC	0.62	0.37	0.47 (0.43 to 0.50)
TBC + PRUE B3	0.70	0.38	0.49 (0.45 to 0.53)
TBC + PRUE B7	0.70	0.40	0.51 (0.47 to 0.55)

With the wider grid, TBC split far fewer fields: its F1 rose to 0.47, level with PRUE B3 with the object procedure (0.44), and the fusion with B3 reached 0.49. PRUE B7 with the object procedure (0.52) remained more accurate than TBC, and its fusion with TBC (0.51) did not improve on it. Much of the gap in the pre-specified comparison therefore came from the thresholds, chiefly the minimum field size: on the tuning tiles, a larger minimum raised the F1 of TBC markedly but hardly changed that of the deep model. Because this analysis was chosen after the results were known, it does not replace the pre-specified comparison.

TBC is therefore a strong method where it was developed and a useful complement to the deep models, but it is not uniformly better than they are, and its accuracy depends on the minimum field size. Where reference fields of the target region exist, train a local model (`vignette("training")`), tune its thresholds over a wide grid and compare TBC, the deep model and their fusion on separate tiles before mapping a large area.

The pretrained models

The pretrained models of the package were fitted with `fb_train()` and `fb_fit_fusion()` on the fitting tiles of California and of California and Washington together, and their thresholds were tuned with `fb_tune()` on the tuning tiles, over a wider grid than in the validation above (minimum field sizes up to 400 pixels instead of 50). On the test tiles, which were not used to fit or tune them:

```
pm <- read.csv(system.file("extdata", "validation", "pretrained_models.csv", package =
 ↪  "fieldborders"))
tab <- data.frame(Model = pm$model, `Test tiles` = sprintf("%s (%d)", pm$region, pm$tiles),
 ↪  Method = pm$method,
                  Fields = pm$reference_fields, Precision = round(pm$obj_precision, 2),
                  Recall = round(pm$obj_recall, 2),
                  `F1 [95 % CI]` = sprintf("%.2f [%.2f, %.2f]", pm$obj_f1, pm$f1_lo, pm$f1_hi),
                  `Recall < 1 ha` = round(pm$recall_s1, 3), check.names = FALSE)
knitr::kable(tab)
```

Model	Test tiles	Method	Fields	Precision	Recall	F1 [95 % CI]	Recall < 1 ha
california	California (54)	TBC	3696	0.78	0.53	0.63 [0.59, 0.67]	0.003
california	California (54)	TBC + PRUE B3	3696	0.82	0.55	0.66 [0.62, 0.70]	0.006
general	California (54)	TBC	3696	0.76	0.54	0.63 [0.59, 0.67]	0.010

Model	Test tiles	Method	Fields	Precision	Recall	F1 [95 % CI]	Recall < 1 ha
general	California (54)	TBC + PRUE B3	3696	0.80	0.54	0.64 [0.60, 0.68]	0.007
general	Washington (44)	TBC	2155	0.54	0.40	0.46 [0.41, 0.50]	0.011
general	Washington (44)	TBC + PRUE B3	2155	0.65	0.42	0.51 [0.48, 0.55]	0.014

With a minimum field size of about 1.1 ha, the pretrained models find almost none of the reference fields smaller than 1 ha, but they split few fields. In Washington, the `"general"` model reached an F1 of 0.457 alone and 0.513 fused with PRUE B3, close to the models fitted and tuned on Washington alone over the same grid (secondary analysis above). The R implementation reproduces the pipeline used for the validation: with the same design, the validation pipeline reached an F1 of 0.633 with TBC and 0.659 with the fusion on the California test tiles, against 0.632 and 0.659 for the `"california"` models of the package.